BCPS 425: PARALLEL AND DISTRIBUTED COMPUTING

Sunyani Technical University (STU) — Complete Official Exam Study Manual & Past Paper Solutions

BCPS 425: Parallel and Distributed Computing — Final Exam Study Guide

> *[!IMPORTANT]*
>
> ***Exam Overview & Strategy***
>
> *This study guide synthesizes all key topics from your course outline, lecture presentations (`INTRO TO PARALLEL AND DIST 1.pptx`, `Cloud Computing Lect 4.pptx`), topic checklist (`exams_topics.txt`), textbook chapter solutions (Coulouris et al. 5th Ed), and **[STU 2023/2024 Solved Past Exam Paper](file:///C:/Users/USER/.gemini/antigravity/brain/6ef44216-f50c-45a0-b3dd-79092ca42930/past_exam_2023_2024_solutions.md)**.*
>
> *Review each module carefully, pay special attention to **definitions, differences, algorithms, formulas, code snippets (e.g. Java RMI)**, and the **Textbook Q&A Bank** extracted directly from your solutions folder.*

Table of Contents

Module 1: High Performance Computing (HPC) & Parallel Architectures

Key Concepts

- **Parallel Computing**: Computing method where multiple processors execute multiple smaller sub-tasks simultaneously on a single computer system to improve speed and execution efficiency.
- **Distributed Computing**: System of multiple autonomous computers connected over a network, coordinating through message passing to achieve a shared objective.
- **High Performance Computing (HPC)**: Aggregation of computing power (supercomputers or computer clusters) to process complex, data-intensive tasks at extremely high speeds.

Feature	Parallel Computing	Distributed Computing
Physical Location	Single machine / tightly coupled chassis	Geographically separated / networked computers

Memory Architecture	Shared memory or fast interconnection bus	Distributed / private memory per node
Communication	Shared variables, bus, shared RAM	Message passing over network (TCP/IP, Sockets)
Primary Goal	High execution speed / throughput	Resource sharing, scalability, fault tolerance

Parallel Architectures & Flynn's Taxonomy

Flynn's classification of computer architectures based on Instruction and Data streams:

- **SISD (Single Instruction, Single Data)**: Traditional single-processor PC.
- **SIMD (Single Instruction, Multiple Data)**: Vector processors, modern GPUs executing identical operations on data arrays.
- **MISD (Multiple Instruction, Single Data)**: Rare; used mainly in fault-tolerant systems (e.g. space shuttle flight controls).
- **MIMD (Multiple Instruction, Multiple Data)**: Multi-core CPUs, distributed clusters, supercomputers.

Speedup & Amdahl's Law

- **Speedup Formula**:

$$ S(p) = \frac{T_1}{T_p} $$

where T_1 is execution time on 1 processor, and T_p is execution time on p processors.

- **Amdahl's Law**: Limits the theoretical speedup when part of the task is strictly sequential ($1-f$):

$$ S(p) = \frac{1}{(1-f) + \frac{f}{p}} $$

Module 2: 3-Tier & Client-Server Architecture

Client-Server Model

A client-server system consists of client processes (requesting services) and server processes (providing services).

```
flowchart LR
    Client1[Client Device / Browser] -->|Request| Server[Application Server]
    Client2[Mobile App Client] -->|Request| Server
    Server -->|Response| Client1
    Server -->|Response| Client2
```

2-Tier vs 3-Tier Architecture

```
flowchart TD
    subgraph "3-Tier Architecture"
    Tier1["Tier 1: Presentation Layer (User Interface / Client)"]
    Tier2["Tier 2: Application / Business Logic Layer (Server)"]
    Tier3["Tier 3: Data Layer (Database / Storage System)"]
    Tier1 <--> Tier2
    Tier2 <--> Tier3
    end
```

- **Presentation Layer (Tier 1)**: User interface (Web browser, Desktop App, Mobile App).
- **Application / Business Logic Layer (Tier 2)**: Processes rules, business logic, authorization, data processing (Web server / Application server).
- **Data Layer (Tier 3)**: Relational/NoSQL Database Management System (DBMS), cloud storage (e.g., MySQL, PostgreSQL, S3).

[!NOTE]

***Advantages of 3-Tier Architecture**:*

* ***Scalability**: Layers can be scaled independently (e.g. add more web servers without touching database).*

* ***Maintainability**: Modifying business logic does not force UI changes.*

* ***Security**: Database is shielded behind application servers rather than exposed to clients.*

Module 3: Concurrency, Race Conditions & The Lost Update Problem

Definitions

- **Concurrency**: Multiple operations or execution threads making progress within overlapping time intervals.
- **Critical Section**: A block of code that accesses shared resources (memory, file, database) and must not be concurrently executed by more than one process.
- **Race Condition**: A flaw where system output depends on the unpredictable timing or sequence of execution threads.

The Lost Update Problem

Occurs when two concurrent transactions read the same data item, perform calculations based on it, and both write their updates back. The second write completely **overwrites** the first user's update.

Step-by-Step Scenario: Airline Seat Booking

- Seat `22F` status = `AVAILABLE`.
- **User A** reads `22F` (reads `AVAILABLE`).
- **User B** reads `22F` (reads `AVAILABLE`).
- **User A** decides to book and writes `BOOKED BY A` to `22F`.
- **User B** (unaware of A's write) writes `BOOKED BY B` to `22F`.
- **Result**: User A's update is **permanently lost**; seat `22F` is double-booked!

Remedies

- **Pessimistic Locking**: Acquire an exclusive lock on the row before reading/updating (`SELECT ... FOR UPDATE`).
- **Optimistic Concurrency Control (Versioning)**: Check version timestamp upon update (`UPDATE seats SET user='B', version=version+1 WHERE id=22F AND version=1`).

Module 4: Deadlocks (Conditions, Prevention, Avoidance & Recovery)

Definition

A **Deadlock** is a state where a set of processes are permanently blocked because each process holds a resource and waits for another resource held by another process in the set.

The 4 Coffman Conditions (Must all 4 hold simultaneously)

- **Mutual Exclusion**: At least one resource must be held in a non-shareable mode (only 1 process at a time).
- **Hold and Wait**: A process holds at least one resource while waiting to acquire additional resources held by other processes.
- **No Preemption**: Resources cannot be forcibly taken away from a process; they can only be released voluntarily.
- **Circular Wait**: A closed chain of processes exists such that P_0 waits for P_1, P_1 waits for P_2, ..., and P_n waits for P_0.

```
flowchart TD
    P1((Process 1)) -->|Requests| R2[Resource 2]
    R2 -->|Held by| P2((Process 2))
    P2 -->|Requests| R1[Resource 1]
    R1 -->|Held by| P1
```

Deadlock Handling Strategies

```
graph TD
    DeadlockHandling[Deadlock Handling Strategies]
    DeadlockHandling --> Prevention[Deadlock Prevention]
    DeadlockHandling --> Avoidance[Deadlock Avoidance]
    DeadlockHandling --> Detection[Detection & Recovery]

    Prevention --> P_Details[Break 1 of 4 Coffman Conditions e.g., enforce total resource ordering]
    Avoidance --> A_Details[Dynamically inspect allocation state e.g., Banker's Algorithm]
    Detection --> D_Details[Periodically check Wait-For graph for cycles & select Victim process to rollback]
```

- **Deadlock Prevention**: Structural rules preventing at least one of the 4 conditions from ever occurring (e.g. impose global ordering on all resources to eliminate Circular Wait).
- **Deadlock Avoidance**: Requires advance knowledge of process resource needs. System evaluates if allocating a resource leads to a **Safe State** (e.g., Dijkstra's **Banker's Algorithm**).
- **Deadlock Detection & Recovery**: System allows deadlocks to occur, periodically runs a cycle detection algorithm on Resource Allocation Graphs, and recovers by **victim selection** (terminating a process or rolling back a transaction).

Module 5: CPU Scheduling Algorithms (Round Robin vs SJF)

1. Round Robin (RR)

- **Type**: Preemptive scheduling based on a fixed time slice (**quantum** \(q\)).
- **Mechanism**: Processes are placed in a FIFO queue. Each process gets \(q\) units of CPU time. If not finished, it is preempted and put at the back of the queue.
- **Pros**: Fair, no process starves, excellent for interactive/time-sharing environments.
- **Cons**: Context-switching overhead if quantum \(q\) is too small.

2. Shortest Job First (SJF)

- **Type**: Non-preemptive (or preemptive as SRTF — Shortest Remaining Time First).
- **Mechanism**: Selects the process with the shortest CPU burst time next.
- **Pros**: Mathematically **minimizes average waiting time**.
- **Cons**: Risk of **starvation** for long-running processes if short jobs keep arriving; requires knowing job burst times in advance.

Summary Comparison Table

Criteria	Round Robin (RR)	Shortest Job First (SJF)
Primary Metric	Responsiveness & Fairness	Minimum Average Wait Time / Throughput
Preemptive?	Yes (Time quantum enforced)	Non-preemptive (SRTF is preemptive version)
Starvation Risk	Zero (No starvation)	High (Long jobs can starve)
Best Used For	Interactive web / multi-user OS	Batch processing with predictable burst times

---

Module 6: Distributed Operating Systems (DOS) & Protocols

Distributed OS (DOS) vs Network OS (NOS)

- **DOS (Distributed OS)**: Presents a single-system image across multiple machines. Location of processors, files, and memory is completely transparent to the user.
- **NOS (Network OS)**: Collection of standalone autonomous computers connected by a network (e.g., Linux workstations on LAN). Users are aware of distinct machine locations.

Key Transparencies in Distributed Systems

- **Access Transparency**: Local and remote resources are accessed using identical operations.
- **Location Transparency**: Users access resources without knowing their physical location on the network.
- **Replication Transparency**: Multiple copies of resources can exist to increase performance without users knowing.
- **Failure Transparency**: Conceals faults, allowing system to recover without user intervention.

Network Protocol Stack & IPC Primitives

- **Transport Layer Protocols**:
- **TCP (Transmission Control Protocol)**: Connection-oriented, reliable, ordered byte-stream, error checking.
- **UDP (User Datagram Protocol)**: Connectionless, lightweight, un-ordered, zero overhead; used for real-time video/audio streaming.
- **Sockets**: Interface for application process communication (`IP Address : Port Number`).

Module 7: Remote Method Invocation (RMI & Java RMI / RI)

Remote Method Invocation (RMI) Concept

RMI is an API and mechanism in object-oriented distributed systems that enables an object on one JVM to invoke methods on an object running in another JVM.

Architecture & Components of Java RMI

```
flowchart LR
    subgraph Client JVM
    ClientObj[Client Program] --> Stub[Stub Proxy]
    end

    subgraph Server JVM
    Skeleton[Skeleton / Server Helper] --> RemoteObj[Remote Object Implementation]
    end

    subgraph RMI Registry
    Reg[(Naming Registry)]
    end

    Stub <==>|Network Communication| Skeleton
    ClientObj -.->|1. Naming.lookup| Reg
    RemoteObj -.->|2. Naming.rebind| Reg
```

- **Remote Interface (RI)**: Java interface extending `java.rmi.Remote`. Every method must declare `throws RemoteException`.
- **Remote Object**: Implementation class extending `UnicastRemoteObject` and implementing the Remote Interface.
- **Stub**: Client-side proxy object that marshals parameters and unmarshals return values over the network.
- **Skeleton / Layer**: Server-side helper that unmarshals incoming parameters, calls actual method, and marshals result.
- **RMI Registry / Naming Service**: Bootstrap lookup service mapping logical names to remote objects (`Naming.rebind("CalculatorService", obj)`, `Naming.lookup("//host/CalculatorService")`).

Java RMI Code Template

1. The Remote Interface (`ComputeService.java`)

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface ComputeService extends Remote {
    int add(int a, int b) throws RemoteException;
}
```

2. The Remote Object Implementation (`ComputeServiceImpl.java`)

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class ComputeServiceImpl extends UnicastRemoteObject implements ComputeService {
    public ComputeServiceImpl() throws RemoteException {
        super();
    }

    @Override
    public int add(int a, int b) throws RemoteException {
        return a + b;
    }
}
```

3. The Server Registration (`Server.java`)

```
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class Server {
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099); // Start RMI registry on default port
            ComputeService service = new ComputeServiceImpl();
            Naming.rebind("rmi://localhost:1099/ComputeService", service);
            System.out.println("RMI Server Ready.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

4. The Client Invocation (`Client.java`)

```
import java.rmi.Naming;

public class Client {
    public static void main(String[] args) {
        try {
            ComputeService service = (ComputeService) Naming.lookup("rmi://localhost:1099/ComputeService");
            int result = service.add(15, 25);
            System.out.println("Remote Calculation Result: " + result);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Module 8: Cloud Computing & Distributed Security / Fault Tolerance

Cloud Computing Service Models

- **IaaS (Infrastructure as a Service)**: Raw virtualized compute, storage, networking (e.g. AWS EC2, Google Compute Engine).
- **PaaS (Platform as a Service)**: Development platform and environment hosted in the cloud (e.g. Heroku, AWS Elastic Beanstalk).
- **SaaS (Software as a Service)**: Ready-to-use software delivered over the internet (e.g. Google Drive, Microsoft 365, Gmail).

Security Pillars

- **Authentication**: Verifying **who** a user/system is (e.g. Passwords, Multi-Factor Authentication, Digital Certificates).
- **Authorization**: Checking **what permissions** an authenticated user has (e.g., Role-Based Access Control).
- **Encryption**: Protecting data confidentiality in transit (TLS/HTTPS) and at rest (AES-256).

Fault Tolerance & Redundancy

- **Fault Tolerance**: The capability of a system to maintain operational state even when hardware or software components fail.
- **Active Redundancy**: All redundant nodes process incoming requests in parallel (instant failover).
- **Passive Redundancy**: Primary node processes requests; secondary backup node takes over only if primary fails heartbeat check.

Module 9: Official Textbook Solution Q&A Bank

(Extracted directly from `Solution/*.pdf` chapter solution files)

Q1: What is the difference between Maybe, At-Least-Once, and At-Most-Once RMI Invocation Semantics? (Chapter 5, Exercise 5.12)

- **Answer**:
- **Maybe**: The client sends a request and does not retransmit if no reply is received. The method may execute **0 times or 1 time**. No fault tolerance mechanisms are applied.
- **At-Least-Once**: The client continuously retransmits requests on timeout until a reply is returned. The remote method executes **1 or more times**. Safe *only* for idempotent operations.
- **At-Most-Once**: The client retransmits on timeout, but the server maintains a **history of past replies** to filter out duplicate requests. The remote method executes **0 times or exactly 1 time**.

Q2: Discuss whether the following operations are Idempotent: (i) Pressing an elevator call button; (ii) Writing data to a fixed file offset; (iii) Appending data to a file. (Chapter 5, Exercise 5.7)

- **Answer**:
- **(i) Pressing an elevator call button**: **Idempotent**. Pressing it once or multiple times leaves the system state identical (button remains illuminated, request registered).
- **(ii) Writing data to a fixed file offset**: **Idempotent**. Repeating the write operation with the same payload at offset `X` results in the exact same file content.
- **(iii) Appending data to a file**: **NOT Idempotent**. Each execution extends the length of the file by adding another payload instance.

Q3: Why does CORBA CDR contain no explicit data-typing, whereas XML payloads include explicit tags? (Chapter 4, Exercise 4.9)

- **Answer**:
- **CORBA CDR** relies on static Interface Definition Language (IDL) files compiled beforehand on both client and server sides. Because both parties know the exact data type order in advance, binary payloads carry zero type tags, maximizing transmission speed and minimizing payload size.
- **XML** is self-describing and dynamic. Data types and names are explicitly embedded within textual tags, providing readability and flexibility at the cost of high bandwidth overhead.

Q4: Explain the 3-Tier Architecture for a distributed web application and discuss its benefits over a 2-Tier architecture. (Chapter 2, Exercise 2.9)

- **Answer**:
- **Tiers**:
- *Presentation Tier*: Browser / Client UI.
- *Application Tier*: Middle-tier application servers executing core business rules.
- *Data Tier*: Relational / NoSQL database storing persistent data.
- **Benefits**:
- **Scalability**: Middle-tier application logic can be scaled horizontally across cluster nodes independently of the database.
- **Maintainability**: Business logic can be updated without modifying or re-deploying client UI code.

- **Security**: Database is completely isolated on an internal private network and shielded from direct public internet requests.

Q5: What are the 3 essential conditions for Distributed Mutual Exclusion? (Chapter 15, Exercise 15.2)

- **Answer**:
- **ME1 (Safety)**: At most one process can execute inside the critical section at any given point in time.
- **ME2 (Liveness)**: Requests to enter and exit the critical section will eventually succeed (no deadlocks or infinite starvation).
- **ME3 (Ordering / Fairness)**: If request \(A\) happened-before request \(B\) according to logical timestamps, access to the critical section is granted in the order \(A\) then \(B\).

Q6: How do Shared (Read) Locks and Exclusive (Write) Locks prevent Lost Updates in Concurrent Transactions? (Chapter 16, Exercise 16.1 & 16.2)

- **Answer**:
- **Shared Lock (S)**: Multiple transactions can simultaneously hold shared locks to read an object. No transaction can write to an object while shared locks are held by others.
- **Exclusive Lock (X)**: Only a single transaction can hold an exclusive lock on an object to perform updates. It blocks all incoming read and write lock requests until committed/released, ensuring no second transaction can overwrite modifications concurrently.

Comprehensive Exam Practice Quiz & Answer Key

Multiple Choice Questions (MCQs)

- **What is the primary goal of parallel computing?**
- A. Reduce local storage
- B. Maximize execution speed by using multiple processors simultaneously
- C. Eliminate local network hardware
- D. Avoid using multi-core processors
- **Correct Answer: B**
- **In Flynn's Taxonomy, modern Graphics Processing Units (GPUs) belong to which class?**
- A. SISD
- B. SIMD
- C. MISD
- D. MIMD
- **Correct Answer: B**
- **Which of the following is NOT one of the 4 Coffman conditions for a deadlock?**
- A. Mutual Exclusion
- B. Hold and Wait
- C. Preemption
- D. Circular Wait
- **Correct Answer: C** *(The actual condition is No Preemption)*
- **The Lost Update Problem is a classic consequence of which issue in distributed systems?**
- A. Unencrypted network traffic
- B. Uncontrolled concurrent access / Race conditions
- C. Deadlock avoidance
- D. Using Shortest Job First scheduling
- **Correct Answer: B**
- **Which scheduling algorithm guarantees that NO process will suffer from starvation?**
- A. Shortest Job First (SJF)

- B. Priority Scheduling
- C. Round Robin (RR)
- D. Non-preemptive LIFO
- **Correct Answer: C**
- **Java RMI uses which client-side object as a local proxy for the remote service?**
- A. Skeleton
- B. Stub
- C. Registry
- D. Dispatcher
- **Correct Answer: B**
- **In a 3-tier system architecture, where is the primary business logic implemented?**
- A. Presentation Layer
- B. Application Layer (Tier 2)
- C. Database Layer
- D. Client Browser
- **Correct Answer: B**
- **Which deadlock handling strategy dynamically evaluates system states (e.g. Banker's Algorithm) before granting resource requests?**
- A. Deadlock Prevention
- B. Deadlock Avoidance
- C. Deadlock Recovery
- D. Deadlock Termination
- **Correct Answer: B**
- **What exception MUST be declared by methods in a Java RMI Remote Interface?**
- A. `NullPointerException`
- B. `RemoteException`
- C. `ClassNotFoundException`
- D. `IOException`
- **Correct Answer: B**
- **Cloud service model providing virtual virtual machines, raw storage, and firewalls is classified as:**
- A. SaaS
- B. PaaS
- C. IaaS
- D. FaaS
- **Correct Answer: C**

Key True/False Questions

- **True/False**: A deadlock can occur even if only 3 of the 4 Coffman conditions are present.
- **Answer: False** *(All 4 conditions must hold simultaneously).*
- **True/False**: Shortest Job First (SJF) scheduling minimizes the average waiting time for a set of jobs.
- **Answer: True**
- **True/False**: Authentication and Encryption refer to the exact same security concept.
- **Answer: False** *(Authentication proves identity; Encryption protects confidentiality).*
- **True/False**: A system can be fault-tolerant without being encrypted.
- **Answer: True** *(Fault tolerance relates to availability; encryption relates to security/privacy).*

- **True/False**: In Java RMI, remote object reference parameters are passed by reference, while non-remote serializable parameters are passed by value.
- **Answer: True**

STU End of Second Semester Examination 2023/2024

BCPS 425: Parallel & Distributed Computing — Official Past Paper Solutions

***Institution**: Sunyani Technical University (STU)*
***Faculty**: Applied Science & Technology* | ***Department**: Computer Science*
***Course Code & Title**: BCPS 425: Parallel & Distributed Computing*
***Examiner**: Adjei-Gyabaa Sylvester Kwasi*
***Time Allowed**: 2½ Hours* | ***Total Marks**: 60 Marks*

Question 1 (18 Marks Total)

(a) State and explain three (3) specific and contrasting examples of the increasing levels of heterogeneity experienced in contemporary distributed systems. (6 Marks)

Model Solution:

Heterogeneity in distributed systems refers to the variety and diversity of system components. Three contrasting examples include:

- **Hardware & Processor Architectures**:
- *Explanation*: Modern distributed systems run across heterogeneous instruction set architectures (ISAs)—such as x86_64 CPUs, ARM64 processors (Apple Silicon/mobile devices), and SIMD GPUs (NVIDIA CUDA). They differ in byte ordering (Big-Endian vs Little-Endian), clock speed, word size (32-bit vs 64-bit), and parallel floating-point processing capabilities.
- **Operating Systems**:
- *Explanation*: Nodes in a cluster or cloud environment execute different operating systems (e.g. Linux distributions, Microsoft Windows Server, macOS, Android/iOS). Each OS provides distinct system calls, process scheduling algorithms, network stack implementations, and file system primitives (POSIX vs NTFS).
- **Programming Languages & Middleware Frameworks**:
- *Explanation*: Distributed components are authored in different languages (Java, Python, C++, Go, Rust). Middleware standards like **Java RMI**, **gRPC (Protocol Buffers)**, **REST/JSON**, and **CORBA CDR** mask this heterogeneity by providing standard serialization formats so objects compiled in different languages can communicate transparently.

(b) A search engine is a web server that responds to client requests to search in its stored indexes and (concurrently) runs several web crawler tasks to build and update the indexes. What are the requirements for synchronization between these concurrent activities? (4 Marks)

Model Solution:

To maintain consistency and high performance, the synchronization requirements between search tasks and web crawler tasks are:

- **Reader-Writer Synchronization (Mutual Exclusion)**:
- Search query tasks act as **Readers** (accessing the inverted index). Crawler tasks act as **Writers** (adding new indexed documents). Multiple search tasks can read concurrently, but crawler updates must write without corrupting reader memory.
- **Atomic Index Swapping / Snapshot Isolation**:
- Rather than locking the active index during updates (which would block user searches), crawlers update an offline index copy. Once complete, an **atomic pointer swap** replaces the live index instantly.
- **Memory Visibility & Cache Coherence**:
- Memory updates made by crawler threads when modifying shared memory index tables must be immediately visible to search threads using volatile barriers or atomic read locks.

(c) Consider two communication services A and B used in asynchronous distributed systems.

- **Service A**: Messages may be lost, duplicated, or delayed, and checksums apply only to headers.
- **Service B**: Messages may be lost, delayed or delivered too fast for the recipient to handle, but those that are delivered arrive in order and with the correct contents.

(i) Describe the classes of failure exhibited by each service. (4 Marks)

(ii) Can service B be described as a reliable communication service? (2 Marks)

Model Solution:

(i) Failure Classes:

- **Service A**:
- **Omission Failures**: Messages can be lost in transit.
- **Timing / Arbitrary Failures**: Messages can be delayed indefinitely or duplicated.
- **Arbitrary (Byzantine) Payload Corruption**: Because checksums apply *only to headers*, the payload content can be corrupted during transmission without detection.
- **Service B**:
- **Omission Failures**: Messages may be dropped due to buffer overflow (delivered too fast for recipient queue).
- **Timing Failures**: Messages may be delayed in the network.
- *Note*: No arbitrary corruption occurs (arrives with correct content) and no reordering occurs (in-order delivery).

(ii) Reliability Assessment of Service B:

- **Answer**: **No, Service B cannot be described as a reliable communication service.**
- *Reason*: A reliable communication service guarantees that **every message sent will eventually be delivered** (validity & integrity without loss). Service B allows messages to be dropped/lost due to buffer overflows without providing automatic retransmission or flow-control acknowledgments.

(d) Consider a pair of processes X and Y that use communication service B to communicate. Suppose X is a client and Y a server, and an invocation consists of a request message from X to Y followed by a reply message from Y to X. Describe the classes of failure that may be exhibited by an invocation. (4 Marks)

Model Solution:

The invocation can exhibit four main classes of failure:

- **Request Message Omission Failure**: The request message sent by Client X is lost or dropped by Service B before reaching Server Y.
- **Server Crash / Execution Failure**: Server Y crashes before or during execution of the requested method.
- **Reply Message Omission Failure**: Server Y executes the request successfully, but the reply message is dropped by Service B on its way back to Client X.
- **Invocation Timing Failure**: Either the request or reply message experiences extreme network delay, causing Client X to time out while Server Y is still processing.

Question 2 (18 Marks Total)

(i) The Internet is far too large for any router to hold routing information for all destinations. Explain how the Internet routing scheme deals with this issue. (4 Marks)

Model Solution:

The Internet handles routing scalability through **Hierarchical Routing** and **CIDR Aggregation**:

- **Autonomous Systems (AS) & Default Routing**:

- The Internet is partitioned into Autonomous Systems (ISPs/Enterprise networks). Core routers inside an AS only maintain detailed routes for local subnets. Outgoing traffic for unknown external destinations is forwarded to a **Default Gateway / Border Router**.
- **CIDR (Classless Inter-Domain Routing) IP Prefix Aggregation**:
- IP addresses are grouped into hierarchical subnets (e.g. `192.168.0.0/16`). A single routing table entry represents millions of individual IP addresses, drastically reducing routing table size in BGP (Border Gateway Protocol) routers.

(ii) In a tabular form describe the work of the protocol layer in Internet applications and the TCP/IP suite over an Ethernet. (10 Marks)

Model Solution:

OSI / TCP/IP Layer	Layer Name	Protocol Examples	Key Functions & Responsibilities
Layer 5 (Application)	Application Layer	HTTP, HTTPS, FTP, DNS, SMTP, Java RMI	Formats end-user application data, defines application protocols, and handles user interaction.
Layer 4 (Transport)	Transport Layer	TCP, UDP	Manages end-to-end communication, port addressing (`IP:Port`), process identification, reliable transmission (TCP retransmissions), flow control, and segmenting.
Layer 3 (Network)	Network / Internet Layer	IP (IPv4, IPv6), ICMP, ARP	Handles packet routing across multiple networks, logical IP addressing, packet fragmentation, and path determination.
Layer 2 (Data Link)	Link / Ethernet MAC Layer	Ethernet (802.3), Wi-Fi (802.11)	Encapsulates IP packets into Ethernet Frames, handles physical MAC addressing (`00:1A:2B:...`), frame error checking (CRC), and media access control.
Layer 1 (Physical)	Physical Layer	Ethernet Cables (Cat6), Fiber, Radio Signals	Transmits raw bit streams (0s and 1s) as electrical voltages, light pulses, or radio waves over physical media.

(iii) Briefly explain the disadvantages of using network-level broadcasting to locate resources in:

(a) a single Ethernet (2 Marks)

(b) an intranet (2 Marks)

Model Solution:

- **(a) Single Ethernet**:
- **Broadcast Storms & CPU Interrupt Overhead**: Every broadcast frame sent to `FF:FF:FF:FF:FF:FF` forces **every single computer interface (NIC)** on the Ethernet switch to interrupt its host CPU to inspect the packet, degrading performance across all machines even if they do not host the resource.
- **(b) An Intranet**:
- **Non-Scalability across Routers**: Routers intentionally block network-level broadcast packets to prevent flooding the global network. Therefore, broadcasting cannot locate resources located across different subnets/routers in an intranet.

(c) Explain the extent to which ethernet multicast improves broadcasting. (2 Marks)

Model Solution:

Ethernet **Multicast** transmits packets to a specific multicast MAC group address (e.g. `01:00:5E:...`). Modern smart network switches (IGMP snooping) deliver multicast packets **only to network interfaces that explicitly registered interest** in that group. Network interfaces not in the group filter out the frame in hardware, avoiding CPU interrupt overhead and saving network bandwidth compared to broadcast.

Question 3 (24 Marks Total)

(a) With the aid of a table, give four (4) differences between Parallel and Distributed Computing. (8 Marks)

Model Solution:

Feature / Dimension	Parallel Computing	Distributed Computing
Physical Architecture	Multiple processors/cores tightly coupled within a single chassis/computer.	Multiple autonomous computers connected over a LAN/WAN network.
Memory Access	Shared Memory space accessible by all processors (or ultra-fast bus).	Distributed / Disjoint Memory; each machine has its own private RAM.
Communication Mechanism	Shared variables, bus, or shared memory locks/semaphores.	Network message passing (Sockets, TCP/IP, REST, RMI).
Fault Tolerance & Scalability	Low fault tolerance (if motherboard/power fails, whole system stops). Limited physical scale.	High fault tolerance (individual node failure does not crash the system). Highly scalable across nodes.

(b) Discuss any four (4) advantages of parallel computing. (4 Marks)

Model Solution:

- **Reduced Execution Time (Speedup)**: Tasks are split across multiple CPU cores, executing simultaneously to complete complex computations in a fraction of the time.
- **Ability to Solve Large-Scale Problems**: Allows handling massive datasets (e.g. genome sequencing, weather simulation) that exceed the capacity of single-threaded processors.
- **High Hardware Resource Utilization**: Fully utilizes modern multi-core CPUs and SIMD GPUs rather than idling processor cores.
- **Cost Efficiency**: Combining multiple commodity processors in parallel is significantly cheaper than building a single ultra-fast monolithic processor.

(c) Parallel computing has many application areas. Name and explain any three (3) of these areas. (3 Marks)

Model Solution:

- **Weather Forecasting & Climate Modeling**: Processing massive differential equations and fluid dynamics models across global atmospheric grid points in real-time.
- **Artificial Intelligence & Machine Learning**: Training deep neural networks (LLMs, computer vision) using parallel matrix multiplications on GPU clusters.
- **Financial Risk Analysis & Computational Finance**: Running parallel Monte Carlo simulations to calculate portfolio risk and option pricing in stock markets.

(d) There are certain technologies working behind the cloud computing platforms making it flexible, reliable, and usable. Explain any three (3) of these technologies. (3 Marks)

Model Solution:

- **Virtualization & Hypervisors**: Hardware abstraction technology (e.g. KVM, VMware) allowing multiple virtual machines (VMs) with distinct OS instances to run on a single physical host server.
- **Containerization (Docker / Kubernetes)**: Lightweight OS-level virtualization that packages applications and dependencies together, enabling fast deployment, scaling, and isolation.
- **Load Balancing & Dynamic Resource Allocation**: Distributes incoming client traffic across server pools automatically, scaling resources up or down based on real-time load.

(e) With the aid of a diagram explain the web service architectural pattern. (2 Marks)

Model Solution:

```
flowchart TD
    Provider[Service Provider] -->|1. Publish WSDL| Registry[(Service Registry / UDDI)]
    Requester[Service Requester / Client] -->|2. Find / Lookup Service| Registry
    Requester <==>|3. Bind & Invoke SOAP/REST APIs| Provider
```

- **Explanation**:
- **Service Provider**: Builds the web service and publishes its interface description (WSDL/OpenAPI) to a Service Registry.
- **Service Registry**: Central repository storing service availability and definitions.
- **Service Requester (Client)**: Discovers the service in the registry, binds to the provider's endpoint, and executes remote API calls using XML (SOAP) or JSON (REST).